

Caching

সহজ কথায় বলতে গেলে, **Caching (ক্যাশিং)** হলো বারবার দরকার হয় এমন তথ্যকে একটি দ্রুতগতির অস্থায়ী মেমোরিতে (Temporary Storage) জমা রাখা, যাতে প্রতিবার আসল সোর্স (যেমন: ডাটাবেস বা সার্ভার) থেকে তথ্য আনতে না হয়।

একটি বাস্তব উদাহরণ দিলে বিষয়টি পরিষ্কার হবে: ধরুন আপনার পড়ার টেবিলের পাশে একটি ছোট ড্রয়ার আছে যেখানে আপনি কলম, ডায়েরি বা হেডফোন রাখেন যা আপনার সবসময় লাগে। কিন্তু আপনার সব বই রাখা আছে পাশের রুমের বড় আলমারিতে। বারবার পাশের রুমে যাওয়ার চেয়ে ড্রয়ার থেকে জিনিস নেওয়া অনেক দ্রুত। এখানে আপনার **ড্রয়ারটি হলো ক্যাশ (Cache)**।

১. ক্যাশিং কীভাবে কাজ করে?

যখন আপনি কোনো অ্যাপ বা ওয়েবসাইটে কিছু সার্চ করেন:

১. সিস্টেম প্রথমে **Cache**-এ চেক করে ওই তথ্যটি সেখানে আছে কি না (**Cache Hit**)।
২. যদি থাকে, তবে ক্যাশ থেকেই সাথে সাথে তথ্যটি দিয়ে দেয়।
৩. যদি না থাকে (**Cache Miss**), তবে সিস্টেম আসল ডাটাবেস থেকে তথ্য নিয়ে আসে এবং ভবিষ্যতে ব্যবহারের জন্য সেটি ক্যাশ-এ জমা রাখে।

২. ক্যাশিং কেন দরকার? (Benefits)

- **দ্রুত লোডিং (Speed):** ক্যাশ থেকে ডাটা মেমোরি (RAM) থেকে আসে, তাই এটি হার্ডডিস্ক বা ডাটাবেস থেকে অনেক গুণ দ্রুত কাজ করে।
- **সার্ভারের চাপ কমানো:** বারবার ডাটাবেসে কুয়েরি করতে হয় না বলে সার্ভারের প্রসেসিং পাওয়ার সাশ্রয় হয়।
- **ব্যান্ডউইথ সাশ্রয়:** একই বড় ফাইল বারবার ডাউনলোড না করে ক্যাশ থেকে ব্যবহারের ফলে ইন্টারনেটের খরচ কমে।

৩. ক্যাশিং-এর প্রকারভেদ (Types of Caching)

ক. Browser Caching (ক্লায়েন্ট সাইড)

আপনার ব্রাউজার (Chrome/Safari) ওয়েবসাইটের লোগো, সিএসএস (CSS) বা ইমেজগুলো আপনার পিসিতে সেভ করে রাখে। ফলে দ্বিতীয়বার ওই ওয়েবসাইটে ঢুকলে সেটি দ্রুত লোড হয়।

খ. CDN Caching

আপনার ওয়েবসাইট যদি আমেরিকায় হোস্ট করা থাকে এবং ভিজিটর যদি বাংলাদেশ থেকে ঢুকে, তবে **CDN (Content Delivery Network)** বাংলাদেশের কোনো কাছাকাছি সার্ভারে ওয়েবসাইটের কপি রেখে দেয়।

গ. Application Caching (Redis/Memcached)

ডেভেলপাররা ডাটাবেসের জটিল কুয়েরিগুলো বারবার না চালিয়ে সেগুলোকে **Redis** বা **Memcached** নামক দ্রুতগতির RAM-ভিত্তিক ডাটাবেসে জমা রাখে।

ঘ. Database Caching

ডাটাবেস নিজেই তার মেমরিতে কিছু কমন কুয়েরি রেজাল্ট সেভ করে রাখে যাতে দ্রুত রেসপন্স করা যায়।

৪. ক্যাশিং-এর প্রধান চ্যালেঞ্জসমূহ

ক্যাশিং যেমন উপকারী, এর কিছু চ্যালেঞ্জও আছে:

- **Cache Invalidation (ক্যাশ ইনভ্যালিডেশন):** যদি আসল ডাটা বদলে যায় (যেমন: পণ্যের দাম ১০০ থেকে ৮০ টাকা হলো), তবে ক্যাশ থেকে পুরনো ডাটা মুছে নতুন ডাটা আপডেট করা একটি কঠিন কাজ। এটি না করলে ইউজার ভুল তথ্য দেখবে।
- **Stale Data:** ক্যাশে অনেক সময় পুরনো বা মেয়াদোত্তীর্ণ ডাটা থেকে যেতে পারে।
- **Cost:** ক্যাশ রাখার জন্য সাধারণত RAM ব্যবহার করা হয়, যা সাধারণ হার্ডডিস্কের চেয়ে অনেক বেশি দামী।

৫. ক্যাশিং-এর জন্য জনপ্রিয় টুলস

1. **Redis:** বর্তমানে সবচেয়ে জনপ্রিয় ডিস্ট্রিবিউটেড ক্যাশিং টুল।
2. **Memcached:** সিম্পল এবং খুব দ্রুতগতির ক্যাশিং সিস্টেম।
3. **Varnish:** এইচটিটিপি (HTTP) এক্সিলারেটর হিসেবে ব্যবহৃত হয়।

সারাংশ: ক্যাশিং হলো সিস্টেমের পারফরম্যান্স বাড়ানোর সবচেয়ে কার্যকর উপায়। আপনি যদি একজন ডেভেলপার হন, তবে আপনার অ্যাপে **Redis** ব্যবহার করে কীভাবে ডাটাবেস কুয়েরি দ্রুত করা যায় তা শেখা উচিত।

⚡ Caching কী?

Cache = Fast temporary storage

সহজ ভাষায় 🙌

বারবার database / API hit না করে

আগেই আনা data **দ্রুত memory থেকে serve করা**

🧠 Real-life example:

- ফোনে Contact list → RAM cache
- বারবার server call লাগে না

? Caching কেন দরকার?

- Performance boost (10x–100x)
- Low latency
- DB load কমে
- Cost কমে
- Scalability বাড়ে

Cache কোথায় বসে?

Client

↓

CDN Cache

↓

Load Balancer

↓

App Cache (Redis)

↓

Database

Cache Types

1. Client-side Cache

- Browser cache
- Mobile cache

Example:

- HTTP cache headers

2. CDN Cache

- CloudFront / Cloudflare
- Image, JS, CSS cache

3. Application Cache

- Redis
- Memcached

- In-memory (LRU)

4. Database Cache

- Query cache
- Buffer pool

🔑 Cache Key Concept

key = user:123:profile

value = JSON data

TTL = 60 sec

🕒 TTL (Time To Live)

Data কতক্ষণ cache এ থাকবে

- Short TTL → fresh data
- Long TTL → fast response

🔄 Cache Invalidation (সবচেয়ে কঠিন অংশ 🐼)

Strategies:

1. TTL expire
2. Write-through
3. Write-back
4. Explicit delete

“There are only two hard things in CS: cache invalidation & naming things”

✳️ Caching Patterns (FAANG)

1. Cache-Aside (Lazy Loading) ★★ ★

App → Cache miss → DB → Cache set → Response

Use: Most common

2. Read-Through

Cache নিজেই DB থেকে আনে

3. Write-Through

DB write এর সাথে cache write

4. Write-Back

Cache first → later DB sync



Redis as Cache

Why Redis?

- In-memory
- Extremely fast
- TTL support
- Pub/Sub
- Persistence optional.

Redis Basic Commands

SET user:1 "Arman"

GET user:1

EXPIRE user:1 60

DEL user:1

Redis + Backend Example (Pseudo)

```
data = redis.get(key)
```

```
if not data:
```

```
    data = db.fetch()
```

```
    redis.set(key, data, ex=60)
```

```
return data
```

Cache Problems

Cache Stampede

Many requests on cache miss

Fix:

- Lock
- Random TTL
- Request coalescing

Cache Penetration

Invalid keys flood

Fix:

- Bloom filter
- Cache null value

Cache Inconsistency

Cache $\neq$ DB

Fix:

- Versioning
- Event-based invalidation

Eviction Policy

- LRU 
- LFU
- FIFO
- Random

Cache Security

- Don't cache sensitive data
- Encrypt if needed
- Access control

Cache in System Design

Example: Social Media Feed

- Redis: feed cache
- CDN: images
- TTL: 30 sec
- Write-through for likes

Interview Must-Know

Q: Cache-aside vs Write-through?

 Lazy vs eager

Q: Why Redis over DB?

 In-memory speed

Q: TTL effect?

👉 Freshness vs performance

Q: Where to cache?

👉 Read-heavy paths

🏆 Best Practices (FAANG)

- ✓ Cache hot data only
- ✓ Measure hit ratio
- ✓ Graceful fallback
- ✓ Avoid huge keys
- ✓ Monitor eviction

🔗 Redis vs Memcached

Redis	Memcached
Persistent	Volatile
Data structures	Simple KV
Replication	No
Slower (still fast)	Faster

🧠 One-line Summary

Cache = speed

TTL = freshness

Invalidation = hardest part